UJI KOMPETENSI LSP SKEMA PROGRAM ANALYST LAPORAN PRAKTIK DEMONSTRASI

NAMA ASESOR

Nurhuda Maulana, S.T, M.T.

NAMA ASESI

William Jenkins Sinaga 2310511132

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI S1 INFORMATIKA

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA

2026

A. Deskripsi sistem

Sistem ini dibuat untuk membantu proses pengelolaan inventaris barang secara terstruktur dan efisien dalam suatu organisasi, seperti sekolah, kantor, atau instansi. Sistem ini memungkinkan pengguna untuk melakukan pencatatan data barang, mengelola peminjaman dan pengembalian, serta menghasilkan laporan secara otomatis.

Selain itu, sistem ini juga dilengkapi dengan fitur manajemen pengguna berbasis role (admin, petugas, dan staff), sehingga setiap pengguna memiliki hak akses yang berbeda sesuai dengan tanggung jawabnya. Dengan adanya sistem ini, diharapkan proses monitoring inventaris menjadi lebih akurat, transparan, dan mudah diakses secara real-time melalui aplikasi berbasis web.

B. Proses pengembangan:

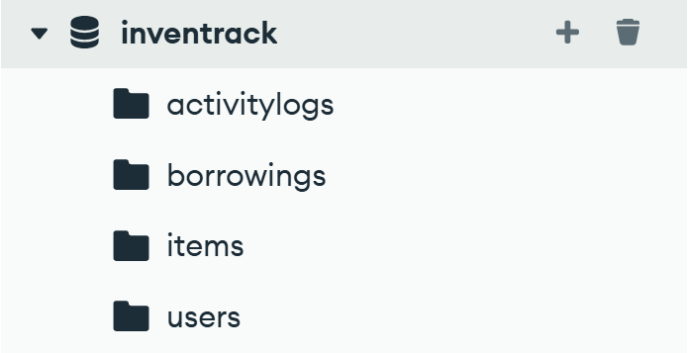
Proses pengembangan sistem dilakukan melalui beberapa tahapan yang terstruktur sebagai berikut:

1. Analisis Kebutuhan

   Pada tahap ini dilakukan identifikasi kebutuhan sistem, seperti fitur pengelolaan barang, peminjaman, laporan, serta sistem autentikasi dan otorisasi pengguna.

2. Perancangan Database

   Database dirancang menggunakan MongoDB dengan beberapa koleksi utama seperti User, Item, Borrowing, dan ActivityLog. Setiap koleksi disusun dengan struktur yang sesuai untuk mendukung relasi data dan kebutuhan sistem.



User

Field	Type
`name`	String
`email`	String
`password`	String
`role`	Enum
`department`	String
`timestamps`	Date

Item

Field	Type
`code`	String
`name`	String
`category`	Enum
`brand`	String
`condition`	Enum
`status`	Enum
`location`	String
`acquisitionDate`	Date
`quantity`	Number
`description`	String
`createdBy`	ObjectId

Borrowing

Field	Type
`item`	ObjectId
`borrower`	ObjectId
`borrowDate`	Date
`returnDate`	Date
`actualReturnDate`	Date
`status`	Enum
`purpose`	String
`notes`	String
`approvedBy`	ObjectId

ActivityLog

Field	Type
`user`	ObjectId
`action`	String
`target`	String
`targetId`	ObjectId
`details`	Mixed
`timestamp`	Date

3. Perancangan Sistem (Design)

   Pada tahap ini dibuat desain arsitektur sistem menggunakan pendekatan MERN Stack (MongoDB, Express.js, React.js, Node.js). Selain itu, dilakukan perancangan tampilan antarmuka (UI/UX) agar user-friendly dan responsif.

4. Implementasi (Coding)

   Proses pengkodean dilakukan dengan membangun frontend menggunakan React dan backend menggunakan Node.js serta Express. API dibuat untuk menghubungkan frontend dan backend, serta mengelola data dari database.

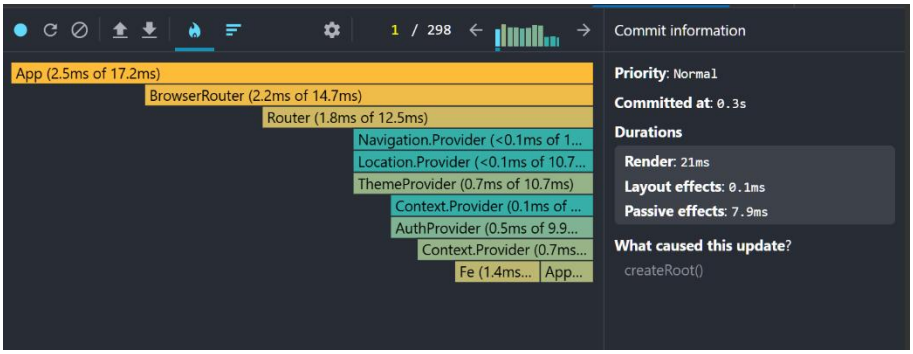
5. Pengujian (Testing)

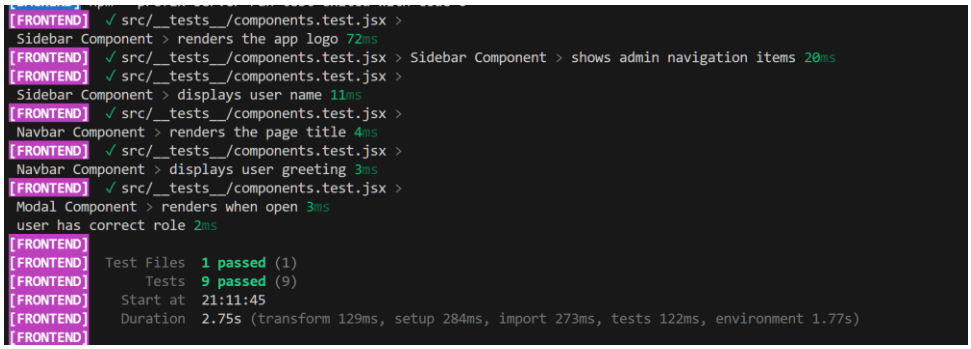
   Sistem diuji untuk memastikan semua fitur berjalan dengan baik, mulai dari pengujian unit, integrasi, hingga pengujian fungsional. Pengujian dilakukan untuk menemukan dan memperbaiki bug atau error.

C. Pengujian

1. Unit Testing
   - Frontend

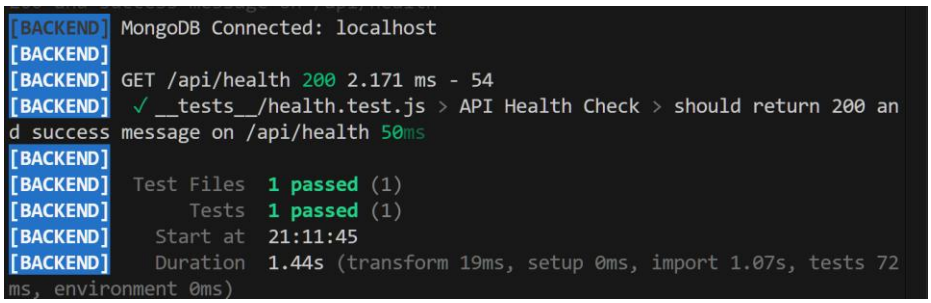
     Pengujian dilakukan menggunakan Vistest, react Testing Library dan JSDOM. Untuk hasilnya semuanya Passed dan berhasil.





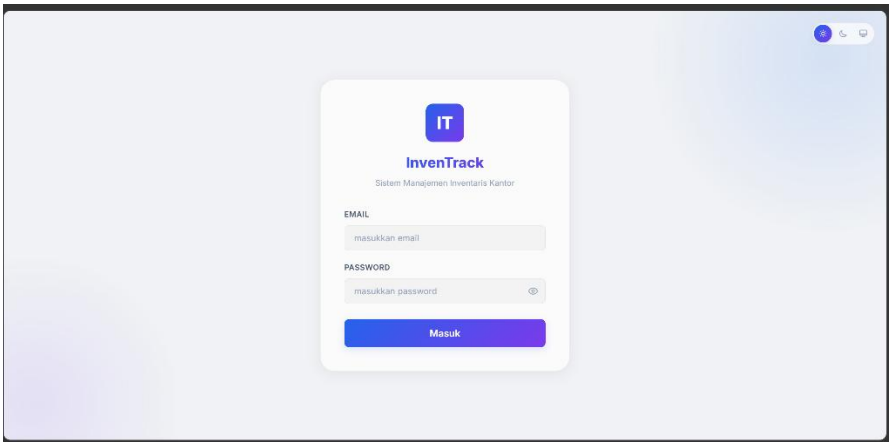
- Backend

  Pengujian pada BackEnd menggunakan Library Morgan untuk melihat kualitas program dan hasilnya baik.

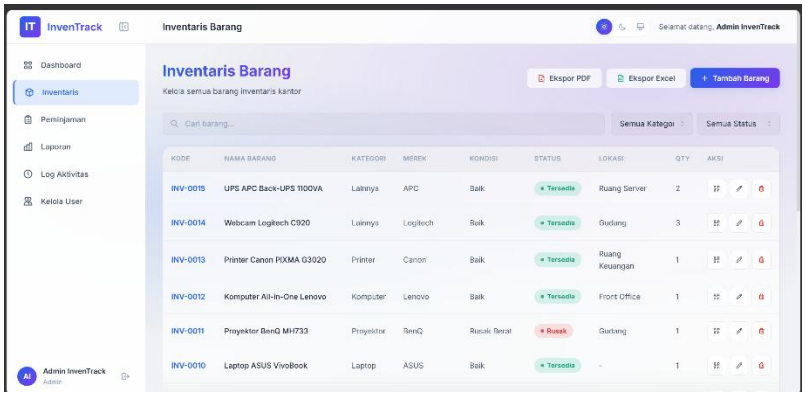


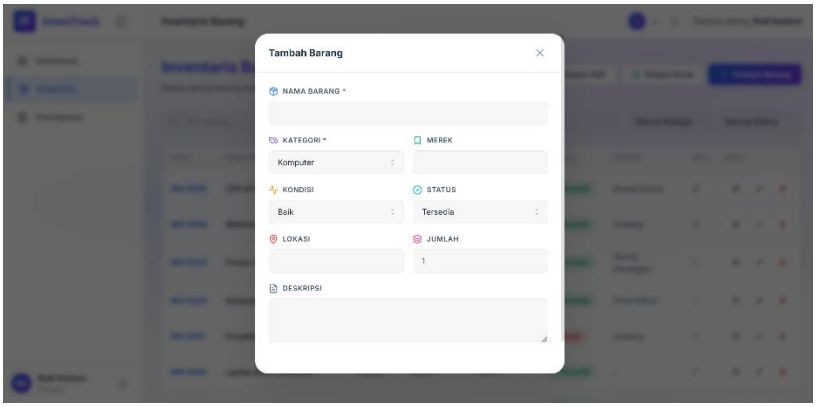
2. Integration Testing

   - Modul Autentikasi & Akses: Menguji alur login pengguna untuk memvalidasi penerbitan token JWT, penyimpanan sesi di Context, dan pengalihan halaman yang sesuai dengan role (Admin/Petugas/Staff). Pengujian juga memastikan middleware berhasil memblokir Staff yang mencoba mengakses URL halaman khusus Admin.

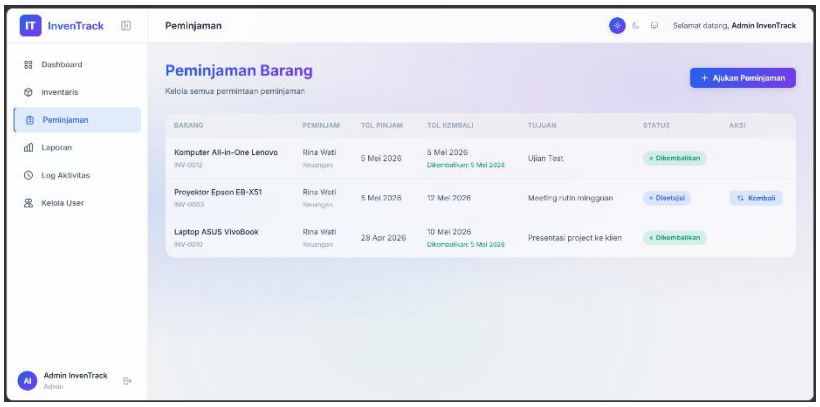


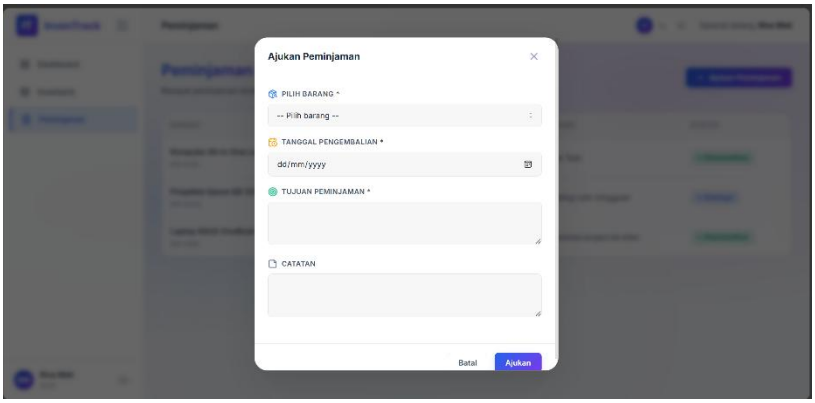
- Modul Inventaris (CRUD): Menguji proses penambahan, pengeditan, dan penghapusan data barang untuk memastikan data tersimpan di MongoDB, muncul pada tabel antarmuka, dan filter pencarian berfungsi secara real-time. Diuji pula sistem proteksi yang menolak penghapusan barang jika statusnya masih "Dipinjam".



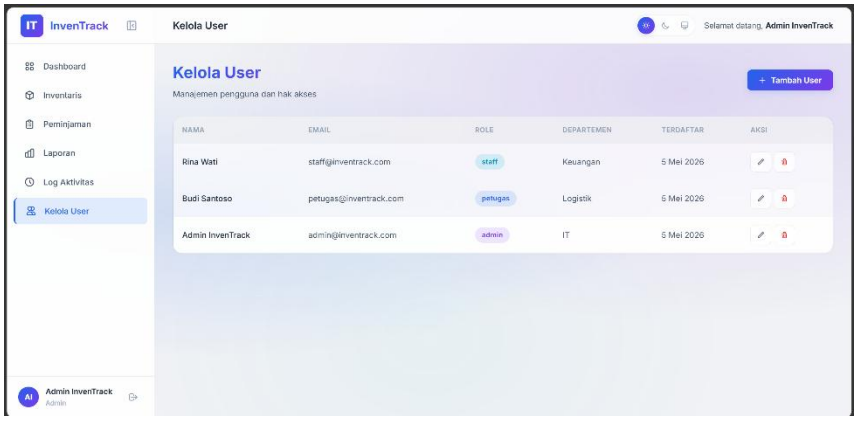


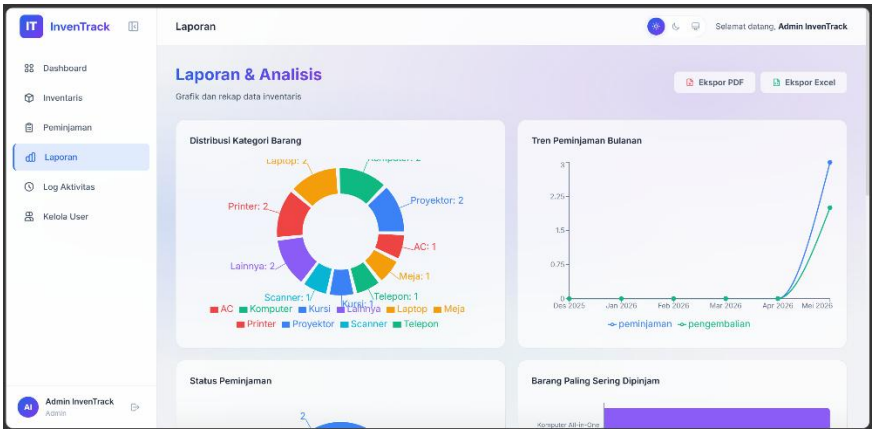
- Modul Peminjaman (Logika Bisnis): Menguji siklus peminjaman secara utuh; mulai dari pengajuan pinjaman oleh Staff (menciptakan record berstatus "Menunggu"), proses persetujuan oleh Petugas (mengubah status menjadi "Dipinjam" dan secara otomatis mengkalkulasi pengurangan stok barang), hingga proses pengembalian (mencatat tanggal kembali aktual dan mengembalikan jumlah stok barang ke angka semula).





- Modul Pengguna & Laporan: Menguji keberhasilan eksekusi penambahan pengguna baru oleh Admin, serta menguji fitur ekspor pelaporan untuk memastikan fail PDF dan Excel berhasil diunduh dengan tarikan data statistik yang sesuai dengan kondisi basis data saat itu.



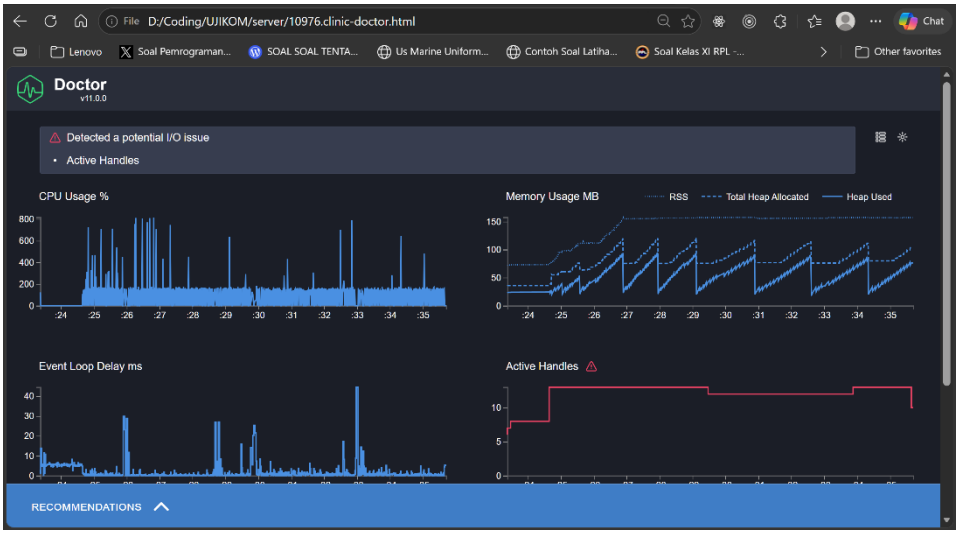


D. Profiling

Pengujian performa (profiling) dilakukan menggunakan Clinic.js Doctor yang dikombinasikan dengan load testing menggunakan Autocannon pada endpoint API. Berdasarkan hasil metrik yang dihasilkan dari grafik, berikut adalah analisis kesehatan server:

1. Memory Usage (Penggunaan Memori)

Grafik "Heap Used" menunjukkan pola "gigi gergaji" (sawtooth pattern) yang sangat sehat dan konsisten. Memori naik secara wajar saat menerima banyak request (beban), lalu langsung turun drastis secara berkala. Hal ini membuktikan bahwa mekanisme Garbage Collection (GC) pada Node.js bekerja dengan sangat optimal dan tidak ditemukan indikasi kebocoran memori (Memory Leak).

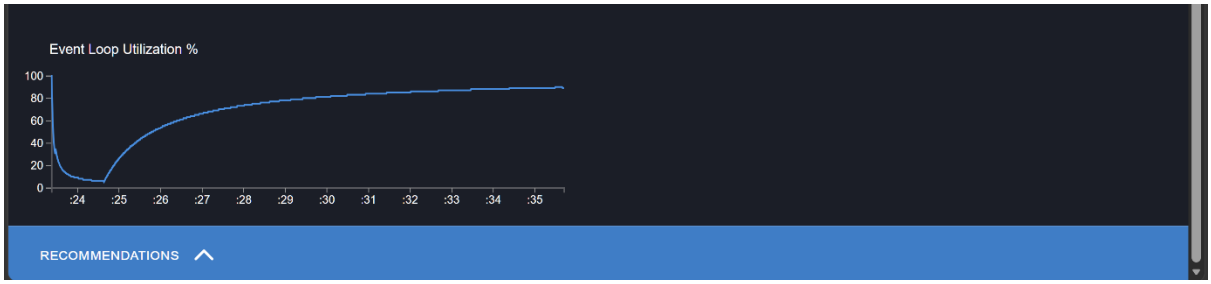


2. CPU Usage (Penggunaan Prosesor)

Grafik CPU menunjukkan adanya lonjakan (spikes) saat load testing berjalan, yang mana sangat normal karena server sedang bekerja keras memproses ratusan request per detik. Tidak ada grafik yang mendatar di posisi atas (100% konstan), membuktikan bahwa server tidak mengalami CPU Bottleneck atau kebuntuan proses logika.

3. Event Loop Delay (Keterlambatan Event Loop)

Keterlambatan maksimal hanya mencapai kisaran ~40ms pada puncak beban tertinggi. Angka ini sangat wajar untuk aplikasi Node.js saat berada di bawah tekanan (stress test). Hal ini mengindikasikan bahwa tidak ada fungsi synchronous yang memblokir (blocking) sistem secara parah, sehingga server tetap responsif melayani request berikutnya.



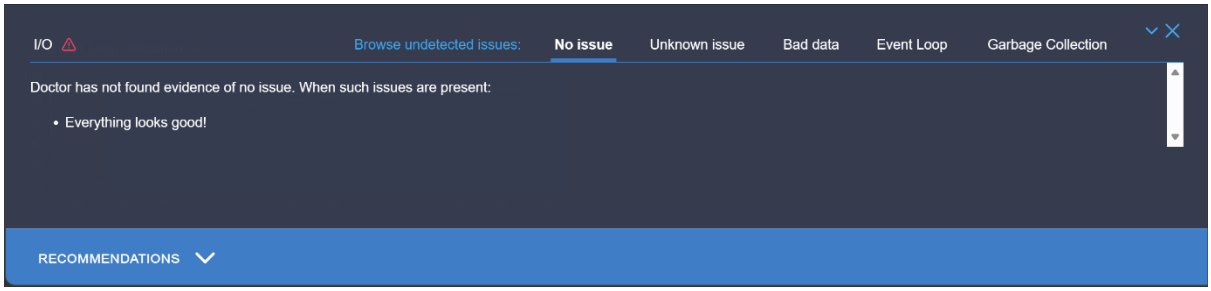
4. Active Handles (Koneksi Aktif)

Grafik "Active Handles" menunjukkan garis yang datar dan stabil di angka ~13 selama load test berlangsung. Ini menandakan bahwa manajemen koneksi (baik itu HTTP

requests dari klien maupun koneksi ke database MongoDB) dikelola dengan baik. Tidak ada koneksi yang menggantung (hanging/zombie connections) yang tertinggal setelah request selesai.

5. Indikasi "Potential I/O Issue"

Sistem mendeteksi peringatan potential I/O issue di bagian atas laporan. Dalam arsitektur aplikasi backend yang terhubung dengan database, ini adalah hal yang wajar dan normal. Peringatan ini sekadar memberitahukan bahwa batas maksimal (limit) kecepatan server saat ini ditentukan oleh antrean operasi Input/Output (waktu tunggu membaca/menulis ke database atau jaringan klien), bukan karena kode program lambat atau memberatkan CPU (CPU-bound). Ini membuktikan bahwa arsitektur asinkron (asynchronous) Node.js telah diimplementasikan dengan benar.



Secara keseluruhan, arsitektur server InvenTrack berada dalam kondisi yang sangat stabil, efisien dalam manajemen pengosongan memori, dan sangat siap untuk menangani beban konkurensi (skala besar) di lingkungan production.

E. Dokumentasi Kode:

Struktur kode pada sistem ini disusun dengan pola MERN Stack agar pemisahan antara frontend, backend, dan database menjadi lebih jelas dan mudah dikelola. Pada sisi frontend, kode diletakkan di folder client/src/ yang berisi komponen, context, pages, services, utils, dan styles. Komponen seperti Navbar, Sidebar, dan ProtectedRoute digunakan untuk membangun antarmuka dan mengatur akses halaman. Sementara itu, AuthContext dan ThemeContext dipakai untuk mengelola state global seperti autentikasi pengguna dan mode tampilan.

```
UJIKOM/
├── client/                     # Frontend (React + Vite)
│   ├── public/                 # Static assets
│   ├── src/
│   │   ├── components/         # Komponen reusable
│   │   │   ├── Modal.jsx       # Komponen modal dialog
│   │   │   ├── Navbar.jsx      # Navigasi atas
│   │   │   ├── ProtectedRoute.jsx  # Route guard (auth + role)
│   │   │   └── Sidebar.jsx     # Navigasi samping
│   │   ├── context/            # React Context (state global)
│   │   │   ├── AuthContext.jsx  # Autentikasi & user state
│   │   │   └── ThemeContext.jsx # Dark/Light mode
│   │   ├── pages/              # Halaman-halaman utama
│   │   │   ├── LoginPage.jsx       # Halaman login
│   │   │   ├── DashboardPage.jsx   # Dashboard utama
│   │   │   ├── InventoryPage.jsx   # Manajemen barang
│   │   │   ├── BorrowingPage.jsx   # Manajemen peminjaman
│   │   │   ├── ReportsPage.jsx     # Laporan & export
│   │   │   ├── UsersPage.jsx       # Manajemen pengguna
│   │   │   └── ActivityLogPage.jsx # Log aktivitas
│   │   ├── services/
│   │   │   └── api.js          # Axios instance & interceptor
│   │   ├── utils/
│   │   │   └── exportUtils.js  # Utility export PDF/Excel
│   │   ├── styles/
│   │   │   └── index.css       # Stylesheet utama
│   │   ├── __tests__/          # Unit tests
│   │   ├── App.jsx             # Root component & routing
│   │   └── main.jsx            # Entry point React
│   ├── index.html              # HTML template
│   ├── vite.config.js          # Konfigurasi Vite
│   ├── eslint.config.js        # Konfigurasi ESLint
│   └── package.json
│
├── server/                     # Backend (Express.js)
│   ├── config/
│   │   └── db.js               # Koneksi MongoDB
│   ├── controllers/            # Logic handler
│   │   ├── authController.js   # Login, register, getMe
│   │   ├── itemController.js   # CRUD barang
│   │   ├── borrowingController.js  # CRUD peminjaman
│   │   ├── reportController.js # Data laporan & statistik
│   │   └── userController.js   # CRUD user & activity log
│   ├── middleware/
│   │   ├── auth.js             # JWT verify & role authorization
│   │   └── errorHandler.js     # Global error handler
│   ├── models/                 # Mongoose schemas
│   │   ├── User.js             # Schema pengguna
│   │   ├── Item.js             # Schema barang inventaris
│   │   ├── Borrowing.js        # Schema peminjaman
│   │   └── ActivityLog.js      # Schema log aktivitas
│   ├── routes/                 # API route definitions
│   │   ├── authRoutes.js       # /api/auth/*
│   │   ├── itemRoutes.js       # /api/items/*
│   │   ├── borrowingRoutes.js  # /api/borrowings/*
│   │   ├── reportRoutes.js     # /api/reports/*
│   │   └── userRoutes.js       # /api/users/*
│   ├── seeders/
│   │   └── seed.js             # Data dummy untuk development
│   ├── .env                    # Environment variables
│   ├── server.js               # Entry point server
│   └── package.json
│
├── docs/                       # Dokumentasi tambahan
├── .gitignore                  # Git ignore rules
└── README.md                   # Dokumentasi utama (file ini)
```

Pada sisi backend, kode disusun di folder server/ dengan pembagian yang terstruktur, yaitu controllers untuk logika proses, models untuk struktur data MongoDB, routes untuk jalur API, serta middleware untuk autentikasi dan penanganan error. File utama seperti server.js menjadi titik awal menjalankan aplikasi, sedangkan koneksi database diatur melalui config/db.js. Struktur ini memudahkan pemeliharaan kode karena setiap bagian memiliki tanggung jawab yang berbeda.

Algoritma yang diimplementasikan dalam sistem ini mencakup beberapa proses utama. Pertama, algoritma autentikasi menggunakan JWT untuk memverifikasi identitas pengguna sebelum mengakses fitur tertentu. Kedua, algoritma role-based access control membatasi hak akses berdasarkan peran pengguna, seperti admin, petugas, dan staff. Ketiga, algoritma CRUD digunakan untuk mengelola data barang, peminjaman, dan pengguna secara sistematis. Selain itu, sistem juga menerapkan auto-generate kode barang, filter pencarian data, serta pencatatan log aktivitas agar data lebih terorganisir dan mudah ditelusuri.

Secara keseluruhan, dokumentasi kode ini menunjukkan bahwa sistem dibangun dengan struktur yang modular, sehingga mudah dikembangkan, diuji, dan diperbaiki ketika diperlukan.

F. Link Github:

Uji_Kompetensi_William_Jenkins_Sinaga